Alla Vostra Mobile App Work Log - September 7, 2026 (Part 1)

Compiled summary for Monday, September 7, 2026 (Part 1) - focus: project-context reread, iPhone native testing direction, App Store/TestFlight versus local-device testing decisions, Xcode setup guidance, Apple Developer and Apple Pay credential constraints, Android Expo Go testing on Galaxy A3 and Galaxy S20 Ultra, Expo Go SDK 54 alignment, native Stripe compatibility shimming for Expo Go, WebP asset conversion and runtime image migration, USB hub/charging-station testing guidance, validation, source commit capture, and current documentation artifact generation for Alla Vostra.

Generated in continuity with the established Alla Vostra worklog cadence: compact opening summary, source-material review, baseline, goals, grouped work-completed sections, validation, files touched, external account/device context, follow-up context, and output-artifact close.

This artifact is named alla_vostra_worklog_september7_part1.pdf because the repository U.S. Eastern-time date used for generation was Monday, September 7, 2026, the user explicitly requested September 7 Part 1, and no earlier September 7 root-level Alla Vostra worklog PDF existed before this documentation pass. It records the work window after the August 24 PDF worklog commit eec913d, through local source commit a2ab388 Prepared native testing and optimized mobile assets. At documentation time, main was ahead of origin/main by one commit, so this PDF records a local committed source state rather than a confirmed remote-synced source state.

Source Material Reviewed

- Reviewed all 60 pre-existing root-level Alla Vostra PDF worklogs from May, June, July, August 1, August 10 Parts 1-3, August 11, August 12 Parts 1-2, August 14, August 16, and August 24 to preserve the project naming pattern, page format, section order, validation language, files-touched inventory, external-account notes, follow-up cadence, and output-artifact close.
- Extracted the full existing worklog set with pdftotext and inspected metadata/titles/page counts with pdfinfo, using the August 24 worklog as the closest style anchor because it is the immediately preceding project PDF and uses the current long-form release/testing documentation shape.
- Rechecked the September 7 project context created earlier in the session after read_project.sh refreshed the app and web snapshots, including the mobile app architecture, Expo SDK 54 dependency set, checkout stack, project routes, native build posture, and prior launch readiness notes.
- Reviewed git history from eec913d through a2ab388, including the latest commit body, name-status output, diff stat, and source-file surface, so this PDF is grounded in the current committed source window rather than only conversational memory.
- Inspected the mobile runtime and configuration files touched by the source window: mob/app.json, mob/eas.json, mob/app.config.js, mob/metro.config.js, mob/package.json, mob/package-lock.json, mob/.env.example, mob/utils/stripePayments.js, and mob/utils/stripeNative.js.

- Inspected the screens/components/data paths whose image sources were migrated to WebP: mob/app/_layout.js, mob/app/index.js, mob/app/products.js, mob/app/aboutus.js, mob/app/shop.js, mob/components/AppHeader.js, mob/components/QuestionOverlay.js, mob/data/products.js, and mob/data/shopOverlayProducts.js.
- Reviewed the committed WebP asset surface, including payment marks, store icons, startup splash assets, product boards, home/about feature mockups, shipping icons, hero backgrounds, and tutorial storyboard/sprite assets.
- Reviewed the user-supplied EAS credential prompt, Apple Developer account questions, Expo Go SDK mismatch report, device list, image-load performance report, and hardware purchasing questions so the worklog captures the practical testing lane as well as code changes.
- Checked the branch state before creating this artifact: main was ahead of origin/main by one local commit, with the new September 7 PDF expected to become the next documentation-only change if committed later.

Baseline Status Before September 7 Part 1

- The August 24 worklog left the Android production release Alla Vostra 1.0.3 / version code 4 in Google Play review, with Data safety changes also in review, live Stripe and PayPal backend infrastructure prepared, and installed-from-Play checkout testing still pending.
- The next active user objective shifted from public Android release submission into native-device testing: first clarifying whether the project was testing or directly publishing, then choosing to run the iPhone build locally through Xcode before App Store or TestFlight publication.
- Xcode was not yet installed at the start of the iPhone-native testing work; xcodebuild still pointed at CommandLineTools, no full Xcode app was present under /Applications, and CocoaPods was not available locally.
- The EAS internal/TestFlight route reached Apple credential prompts when the user declined Apple account login, which confirmed that Apple Developer credentials would be needed for managed internal distribution credential generation or manual credentials.json setup.
- The project already used @stripe/stripe-react-native for payment entry, Apple Pay, Google Pay, and card flows, which meant plain Expo Go could not safely be treated as a complete checkout test environment without a native development or store build.
- The user had two Android devices available for interim Expo Go testing while Xcode downloaded: a Galaxy A3 and a Galaxy S20 Ultra. The Galaxy A3 initially had an Expo Go build for SDK 57 while the project required Expo SDK 54.
- Both Android devices later loaded the app through Expo Go, but large images took several minutes to appear, making image format, asset size, bundling, and tunnel delivery performance the immediate practical bottlenecks.
- The user explicitly requested conversion to .webp while keeping every original image file in its existing directory, so the source work needed to add optimized siblings and migrate active imports without deleting the PNG/JPG originals.
- Hardware planning was still in progress: the user wanted a charging/connecting station for multiple phones, but the testing need separated into power-only charging, USB data/debugging, and network/tunnel performance considerations.

Goals for September 7 Part 1

- Clarify that the immediate path was native testing first, not direct App Store or TestFlight publication.
- Prepare the Expo app for iPhone native testing and later TestFlight/store workflows without forcing the user into paid Apple account steps during this session.
- Preserve a path for Apple Pay in real builds while giving local Xcode testing an Apple-Pay-disabled path that avoids merchant entitlement blockers.
- Make Expo Go usable enough for Android visual/layout testing on the Galaxy A3 and Galaxy S20 Ultra despite the app containing Stripe native checkout dependencies.
- Align the Galaxy A3 to Expo Go SDK 54 so it can open the SDK 54 Alla Vostra project instead of failing on a newer Expo Go runtime mismatch.
- Reduce perceived and actual mobile image-load cost by converting active PNG/JPG assets to WebP while keeping originals untouched in place.
- Keep the existing Alla Vostra UI, checkout flow, tutorial behavior, hero/header behavior, product data, and screen layouts intact while changing only the asset format and native-testing support surfaces.
- Restart the Expo Go testing session after the asset migration so both Android phones could pick up the new bundle and WebP image references.
- Evaluate multi-device hardware purchases according to the real development need: charge phones reliably, connect data-capable devices to the Mac when needed, and avoid mistaking charging-only ports for USB debugging/data ports.

Work Completed - Testing Direction and iPhone Native Path

- Clarified that the project should test the app natively first rather than directly publishing to the App Store or relying only on TestFlight.
- Explained the role of Xcode for a physical iPhone: it installs, signs, launches, and logs the native iOS app on the connected phone, not only an emulator/simulator on the Mac.
- Documented the practical Xcode path for after installation: select the full Xcode developer directory, accept the Xcode license, connect the iPhone by cable, trust the Mac, open the generated workspace, choose the device, set a team, and run the build.
- Added iOS bundle metadata in mob/app.json with bundleIdentifier com.allavostra.app, buildNumber 1, supportsTablet false, and ITSAppUsesNonExemptEncryption false.
- Added expo-dev-client to the mobile dependency set so the project can produce native development builds that behave more like the real installed app than Expo Go while still using the Expo workflow.
- Added a TestFlight EAS build profile and submit profile in mob/eas.json so the project has a named store-distribution lane ready when Apple Developer credentials are available.
- Kept the user on the local Xcode/native testing lane while Xcode downloaded, rather than forcing EAS credential generation after the Apple account prompt blocked internal distribution automation.
- Preserved existing Android production identity and package state while adding iOS-oriented native testing configuration.

Work Completed - Apple Developer, Apple Pay, and Stripe Entitlement Handling

- Clarified the Apple account split: a free Apple ID can generally be used for limited local device testing, while App Store/TestFlight distribution and production capabilities require paid Apple Developer Program enrollment.
- Identified Apple Pay as the most likely native iPhone testing blocker because the Stripe plugin merchantIdentifier path requires Apple merchant capability/entitlement setup for a full Apple Pay-enabled build.
- Added mob/app.config.js to conditionally remove the Stripe plugin merchantIdentifier when ALLA_VOSTRA_DISABLE_APPLE_PAY_ENTITLEMENT=1 is supplied during local prebuild/testing.
- Added EXPO_PUBLIC_DISABLE_APPLE_PAY guidance to mob/.env.example so local app runtime behavior can deliberately avoid exposing an Apple Pay merchant identifier during Apple-Pay-disabled builds.
- Updated mob/utils/stripePayments.js so stripeMerchantIdentifier resolves to an empty string when EXPO_PUBLIC_DISABLE_APPLE_PAY=1 is set, while preserving the existing merchant identifier path for paid/developer/production builds.
- Preserved the production Stripe and platform-pay architecture; the entitlement bypass was added as a local testing control rather than a removal of Apple Pay from the long-term app plan.
- Kept sensitive Stripe, PayPal, Vercel, and Apple credential values out of source and out of the worklog while documenting the credential categories and blockers.

Work Completed - Android Expo Go Device Testing Support

- Added mob/utils/stripeNative.js as a runtime-aware shim around @stripe/stripe-react-native so Expo Go can load visual/test sessions without trying to initialize unavailable native Stripe modules.
- Updated mob/app/_layout.js to import StripeProvider through the new stripeNative wrapper instead of directly from @stripe/stripe-react-native.
- Updated mob/app/shop.js to import CardForm, PlatformPay, usePlatformPay, and useStripe through the same wrapper so card and wallet surfaces remain wired for native builds while Expo Go gets safe fallbacks.
- Made the Expo Go fallback return a clear Native Stripe checkout requires a development or store build message for native checkout actions, while keeping visual navigation and non-native layout review available.
- Guided the Galaxy A3 SDK mismatch repair by pointing the user to Expo Go 54 APK builds and noting that the newer Expo Go install should be removed and Play Store auto-update disabled before installing the SDK 54 APK.
- Kept the Galaxy S20 Ultra on the same SDK 54 Expo Go project path so the old phone and newer phone could be compared against the same bundle and asset set.
- Restarted Expo Go with a clean tunnel after the WebP migration so the Android devices could load the new image references through the current Metro session.
- Treated Expo Go as a visual/device-behavior lane only; complete Stripe, Apple Pay, Google Pay, PayPal capture, and App Store-like native validation remain development-build or store-build responsibilities.

Work Completed - WebP Asset Conversion and Runtime Migration

- Converted the mobile raster image set under mob from PNG/JPG/JPEG to WebP siblings while excluding node_modules, generated native android/ios directories, and .expo cache folders.
- Preserved the original image files in their existing directories, matching the user request to keep original PNG/JPG assets available beside the new WebP files.
- Added 151 committed WebP assets across the mobile root, payment asset folder, store asset folder, tutorial storyboard folders, animated layer assets, cleaned frame assets, smooth sprite atlas, and generated smooth sprite frames.
- Added mob/metro.config.js to include webp in Metro resolver asset extensions so Expo can bundle the converted assets reliably.
- Updated startup splash imagery in mob/app/_layout.js from PNG to WebP, including the Play Store orange gradient and app icon startup art.
- Updated shared header imagery in mob/components/AppHeader.js from PNG to WebP, including the default hero, no-cheeseboard hero, and products hero image.
- Updated the tutorial Got It sprite source in mob/components/QuestionOverlay.js from the PNG sprite atlas to the WebP sprite atlas.
- Updated Home, Products, About Us, and Shop image imports to WebP for feature mockups, board/product art, shipping preview icons, wallet marks, confirmation imagery, and shop hero background imagery.
- Updated product data and shop overlay product data to use WebP product-board images so repeated product surfaces now reference the optimized assets consistently.
- Confirmed active app/component/data/util source no longer contained PNG/JPG/JPEG require calls after the migration, aside from the intentional Stripe native package require that is unrelated to image assets.
- Reduced several important runtime assets substantially: background1.webp landed at roughly 96.6 KB, background1_no_cheeseboard.webp at roughly 80 KB, product-board WebPs around 169-172 KB each, and the tutorial smiley sprite WebP around 959 KB.

Work Completed - Hardware and External Testing Guidance

- Explained that running phones from USB into the Mac can improve native debugging, installation, logs, ADB/Xcode workflows, and device charging, but it does not automatically make Expo Go faster if Expo Go is still loading JavaScript and assets through a network tunnel.
- Separated the hardware requirement into two categories: powered charging for keeping many phones alive during QA, and real USB data hub functionality for Mac-connected debugging or native installation workflows.
- Confirmed that the previously considered Acer 10 Gbps USB-C hub looked suitable for data/ADB-style testing if its downstream ports and cables support data, while noting that its power delivery may primarily charge the host Mac rather than every phone as a multi-phone charging station.
- Evaluated the Kakyanill 100W charging-station/power-strip style device as useful for powering phones and desk equipment, but not sufficient as the Mac-connected data hub because its USB ports appear to be charging ports rather than upstream/downstream USB data ports.

- Recommended alternative budget hub options under or near the preferred price range for multi-phone desk testing, while keeping the caveat that a data-capable hub and data-capable cables are required for native Android/iPhone development workflows.
- Left the practical device-lab guidance clear: use the charging station for power, use a powered data hub for Mac debug/install/logging, and use local native builds or LAN/direct-device workflows when Expo Go/tunnel loading remains too slow.

Validation and Checks Performed

- Ran npx expo-doctor in mob after the native-testing and WebP changes; all 18 checks passed with no issues reported.
- Ran an Android export smoke test after the WebP migration; the bundle completed successfully and included active WebP asset references.
- Verified the key generated asset sizes after conversion so the performance work had concrete evidence instead of only a file-extension change.
- Searched the app, component, data, and util source trees for remaining active PNG/JPG/JPEG require calls after the migration and confirmed active image imports were moved to WebP.
- Checked git diff and git show output for a2ab388 to confirm the committed source window included 171 changed files, 151 new WebP assets, native-testing config, Stripe shim work, package updates, and refreshed project snapshots.
- Checked git status --short --branch before this documentation pass and confirmed main was ahead of origin/main by one local source commit.
- Did not complete a native iPhone Xcode run during this source window because the full Xcode install was still downloading.
- Did not complete a TestFlight/internal-distribution build because Apple Developer credential handling was deferred and the user chose the local Xcode testing path first.
- Did not treat Expo Go as full checkout validation because Stripe native checkout requires a development build or store build for complete native module coverage.
- Did not run a real live Stripe charge, Apple Pay payment, Google Pay payment, PayPal capture, or installed-from-App-Store/TestFlight purchase during this Part 1 testing-preparation window.

Files Touched During the September 7 Part 1 Source Window

- alla_vostra_APP_PROJECT_SNAPSHOT.txt and alla_vostra_WEB_PROJECT_SNAPSHOT.txt - refreshed project snapshots after the September 7 context pass and source changes.
- mob/app.json - added iOS bundle/build metadata, disabled tablet support for iPhone-focused testing, and declared non-exempt encryption status for Apple distribution metadata.
- mob/eas.json - added the testflight build profile and submit profile for future iOS store-distribution work.
- mob/package.json and mob/package-lock.json - added expo-dev-client and refreshed the dependency lockfile for native development build support.
- mob/app.config.js - added the conditional Apple Pay entitlement/merchantIdentifier removal path for local iPhone testing without Apple Pay capability setup.
- mob/.env.example and mob/utils/stripePayments.js - documented and implemented the EXPO_PUBLIC_DISABLE_APPLE_PAY runtime flag.

- mob/utils/stripeNative.js - added the Expo Go/native-build bridge that keeps Stripe native APIs available in native builds while providing safe Expo Go fallbacks.
- mob/app/_layout.js - switched StripeProvider to the runtime-aware wrapper and moved startup splash assets to WebP.
- mob/app/shop.js - switched Stripe card/platform-pay imports to the wrapper and moved shop hero, shipping, wallet, and confirmation images to WebP.
- mob/app/index.js, mob/app/products.js, mob/app/aboutus.js, mob/components/AppHeader.js, mob/components/QuestionOverlay.js, mob/data/products.js, and mob/data/shopOverlayProducts.js - migrated active page, header, tutorial, and product image requires to WebP.
- mob/metro.config.js - added WebP to Metro asset extensions for Expo bundling.
- mob/assets/payments/*.webp - added WebP payment marks for Apple Pay, Google Pay, and PayPal.
- mob/assets/store/*.webp - added WebP store/icon/splash assets, including adaptive foreground, app icon, feature graphic, Play Store orange gradient, splash logo, and splash orange icon.
- mob/assets/tutorial/smiley-got-it-storyboard/**/*.webp - added WebP tutorial source frames, cleaned frames, animated layer assets, the smooth smiley_got_it_sprite atlas, and diagnostic smooth sprite frame siblings.
- mob/*.webp - added WebP siblings for mobile root imagery, including hero backgrounds, product boards, home/about feature mockups, shipping icons, SoFlo art, taste/passion/convenience imagery, and retained original PNG/JPG files beside them.

External Configuration and Device State

- EAS CLI was installed globally by the user and the user was already logged in as prahlin1 before the managed iOS credential prompt appeared.
- The EAS iOS internal distribution path requested Apple Developer login after remote iOS credentials were selected and Apple account login was initially declined, which led to the decision to pause EAS credential generation and use Xcode local-device testing first.
- Xcode was downloading during this work window, so iPhone native testing preparation happened in source/config and procedural guidance rather than through an actual device run.
- The Galaxy A3 and Galaxy S20 Ultra were the active Android physical devices for Expo Go testing while waiting for Xcode.
- The Galaxy A3 required Expo Go SDK 54 alignment instead of SDK 57, and the direct Expo Go 54 APK link was provided so the old phone could load the SDK 54 project.
- The Expo Go tunnel was restarted after the WebP conversion with a clean Metro cache so the Android phones could request the updated bundle and optimized image assets.
- The hardware purchase guidance intentionally distinguished charging products from data hubs because a charging station can keep phones powered but cannot replace Xcode/ADB data connectivity unless it exposes true USB data ports to the Mac.
- No Apple Developer Program enrollment, App Store Connect submission, TestFlight build upload, Play Console release update, Vercel production variable change, Stripe dashboard change, PayPal dashboard change, or Postmark configuration change was completed during this Part 1 source window.

Follow-up Context for Future Sessions

- When Xcode finishes installing, switch the active developer directory to /Applications/Xcode.app/Contents/Developer, accept the Xcode license, and run the app on the physical iPhone through the generated iOS workspace or Expo dev-client workflow.
- For local iPhone testing without Apple Pay capability, use ALLA_VOSTRA_DISABLE_APPLE_PAY_ENTITLEMENT=1 during iOS prebuild and EXPO_PUBLIC_DISABLE_APPLE_PAY=1 during the dev-client Metro session so the app avoids merchant entitlement blockers.
- For full App Store-like iPhone checkout testing, enroll or use the paid Apple Developer account, configure the app/bundle/team/merchant capability path, and test Apple Pay only in a native development/TestFlight/store build.
- Keep Expo Go testing focused on layout, navigation, image loading, tutorial flow, and general visual behavior; do not count Expo Go as proof that Stripe card entry, Apple Pay, Google Pay, or native wallet checkout is production-ready.
- If the Galaxy A3 or Galaxy S20 Ultra still load images slowly after the WebP migration, compare tunnel versus LAN/direct local network performance, clear Expo Go cache, force reload the bundle, and consider a native development build for more realistic asset behavior.
- If committing the September 7 PDF, follow the recent project pattern with a documentation-only commit that adds alla_vostra_worklog_september7_part1.pdf after the source commit a2ab388.
- Push the local a2ab388 source commit and the future PDF worklog commit when ready, because main was ahead of origin/main by one source commit at the time this artifact was generated.
- Review .easignore and future release-archive contents before an iOS or Android store build if WebP-only runtime assets replace PNG paths in production; the store archive must include every referenced WebP asset.
- Before public iOS distribution, retest the full installed app path: launch, splash, tutorial, swipe overlay, Home, Products, About Us, Contact, Shop, cart, delivery details, payment method selection, Stripe card setup, wallet availability, PayPal path, confirmation, and email delivery.
- Continue treating live secrets as external configuration only; do not paste Stripe, PayPal, Apple, Vercel, or Postmark secret values into chat, snapshots, commits, or worklog artifacts.

Output Artifact

- Created alla_vostra_worklog_september7_part1.pdf at the repository root as the latest customary PDF worklog for the Alla Vostra project.
- Used the established root-level worklog filename pattern with the requested Part 1 suffix because this was the first September 7 documentation artifact and the user explicitly asked for September 7 Part 1.
- Preserved the current letter-size PDF convention, footer format, concise opening summary, section ordering, bullet-heavy documentation style, validation language, files-touched detail, external device/account context, follow-up section, and output-artifact close.

- This PDF records the September 7 native-testing preparation work, Android Expo Go support, SDK 54 device alignment, WebP asset migration, hardware guidance, Expo validation, and local source state through a2ab388 Prepared native testing and optimized mobile assets.